

A Practical Guide to K-means clustering

TOPIC -- K-MEANS CLUSTERING

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

-- 01 --

The primary drawback of standard k-means++ is its inherent sequential nature. To find k initial centers, the algorithm must make k separate passes over the data, as the selection of each new center depends on the previous choices. This becomes prohibitively slow when clustering massive datasets into a large number of clusters. To overcome these limitations, researchers developed k-means||, a parallel version of k-means++. Instead of sampling a single point per pass, k-means|| uses an oversampling factor $\Omega(k)$ to sample multiple points in each round. The algorithm drastically reduces the number of passes required, obtaining a nearly optimal set of centers with a time complexity of $O(\log n)$. In practice, as few as five rounds are needed to reach a high-quality solution. After several rounds, the algorithm typically has $O(\log n)$ intermediate centers. These are then assigned weights based on how many points are close to them and re-clustered, often using standard k-means++, to reduce the final set to exact k centers.

-- 02 --

$$\arg \min_{S} \sum_{i=1}^k \sum_{x, y \in S_i} \|x - y\|^2$$

-- 03 --

Description Given a set of observations $(x_1, x_2, \dots, x_n)$, where each observation is a d -dimensional real vector, k-means clustering aims to partition the n observations into k ($\leq n$) sets $S = \{S_1, S_2, \dots, S_k\}$ so as to minimize the within-cluster sum of squares (WCSS) (i.e. variance). Formally, the objective is to find:

-- 04 --

Proper initialization is very important when it comes to finding global optimum in a clustering algorithm. Random initialization is often used for k means clustering, but this can result in poor cluster quality. This is why the k-means++ initialization algorithm was developed. The algorithm attempts to find initial centers that are as close to possible to the global optimum, reducing computing time and producing more accurate clusters. The k-means++ algorithm works by spreading out the initial cluster centers. It selects the first center uniformly at random from the dataset. Each subsequent center is chosen stochastically, with the probability of a point being selected proportional to its contribution to the overall error. This method achieves an $O(\log k)$ approximation of the optimum solution.

-- 05 --

Initialization methods Commonly used initialization methods are Forgy and Random Partition. The Forgy method randomly chooses k observations from the dataset and uses these as the initial means. The Random Partition method first randomly assigns a cluster to each observation and then proceeds to the update step, thus computing the initial mean to be the centroid of the cluster's randomly assigned points. The Forgy method tends to spread the initial means out, while Random Partition places all of them close to the center of the data set. According to Hamerly et al., the Random Partition method is generally preferable for algorithms such as the k-harmonic means and fuzzy k-means. For expectation maximization and standard k-means algorithms, the Forgy method of initialization is preferable. A comprehensive study by Celebi et al., however, found that popular initialization methods such as Forgy, Random Partition, and Maximin often perform poorly, whereas Bradley and Fayyad's approach performs "consistently" in "the best group" and k-means++ performs "generally well".

-- 06 --

Shared Memory Model (OpenMP): This approach divides the work between CPU threads for the reassignment step of the process. These threads independently calculate distances to centroids. Partial sums and counts from each thread are aggregated to compute the new centroids for the next iteration. GPU Parallelization (OpenACC): Unlike the OpenMP implementation, the OpenACC program's parallel directive is called at each necessary step instead of at the very beginning. This exploits the SIMT (Single Instruction, Multiple Threads) architecture of GPUs to perform thousands of distance calculations simultaneously. We see that in both parallel versions; efficiency is greatly increased for large datasets. This speedup is significant in the OpenACC implementation, due to the massively parallel nature of GPU architectures.

-- 07 --

k-means implicitly assumes that the ordering of the input data set does not matter. The bilateral filter is similar to k-means and mean shift in that it maintains a set of data points that are iteratively replaced by means. However, the bilateral filter restricts the calculation of the (kernel weighted) mean to include only points that are close in the ordering of the input data. This makes it applicable to problems such as image denoising, where the spatial arrangement of pixels in an image is of critical importance.

-- 08 --

Recent developments Recent advancements in the application of k-means clustering have explored the integration of k-means clustering with deep learning methods, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), to enhance the performance of various tasks in computer vision, natural language processing, and other domains.

-- 09 --

Assignment step Hartigan and Wong's method starts by partitioning the points into random clusters $\{S_j\}_{j \in \{1, \dots, k\}}$. Update step Next it determines the $n, m \in \{1, \dots, k\}$ and $x \in S_n$ for which the following function reaches a maximum $\Delta(m, n, x) = \phi(S_n) + \phi(S_m) - \phi(S_n \setminus \{x\}) - \phi(S_m \cup \{x\})$. For the x, n, m that reach this maximum, x moves from the cluster S_n to the cluster S_m . Termination The algorithm terminates once $\Delta(m, n, x)$ is less than zero for all x, n, m . Different move acceptance strategies can be used. In a first-improvement strategy, any improving relocation can be applied, whereas in a best-improvement strategy, all possible relocations are iteratively tested and only the best is applied at each iteration. The former approach favors speed, whether the latter approach generally favors solution quality at the expense of additional computational time. The function Δ used to calculate the result of a relocation can also be efficiently evaluated by using equality